00 01 02 03 **04** 05 06 07 08 09 0A 0B 0C 0D

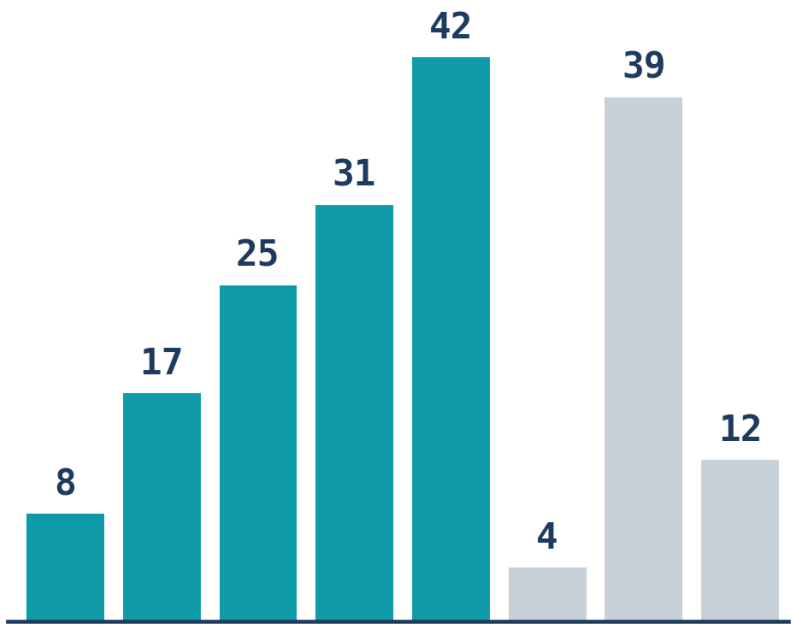
ALGORİTMA VE PROGRAMLAMA – II

HAFTA 05

Temel Sıralama Algoritmaları

Seçmeli, Kabarcık ve Araya Ekleme Sıralaması

MODÜL II · ARAMA VE SIRALAMA



araya ekleme · i = 4 sonrası

Dr. Öğr. Üyesi Mehmet Sevri · RTEÜ Bilgisayar Mühendisliği · 2026–2027 Bahar

14 HAFTALIK YOLCULUK

Yol haritasında neredeyiz?

H1 Doğruluk ve Ölçüm	**H2** Asimptotik Analiz	**H3** Özyineleme ve Geri İzleme	**H4** Arama Algoritmaları	**H5** **Temel Sıralama**	**H6** Böl ve Fethet	**H7** Öbek ve Doğrusal Sıralama
0x00	0x01	0x02	0x03	0x04	0x05	0x06

H8 Dinamik Dizi ve Bağlı Liste	**H9** Yığın ve Kuyruk	**H10** Karma Tabloları	**H11** Graflar: BFS ve DFS	**H12** Açgözlü Algoritmalar	**H13** Dinamik Programlama	**H14** Sentez ve Final Projesi
0x07	0x08	0x09	0x0A	0x0B	0x0C	0x0D

Vize: H1–H7 · Ara proje: H7 (teslim: H8 başı) · Final projesi: H14

TAHMİN ET

Geçen haftadan üç soru

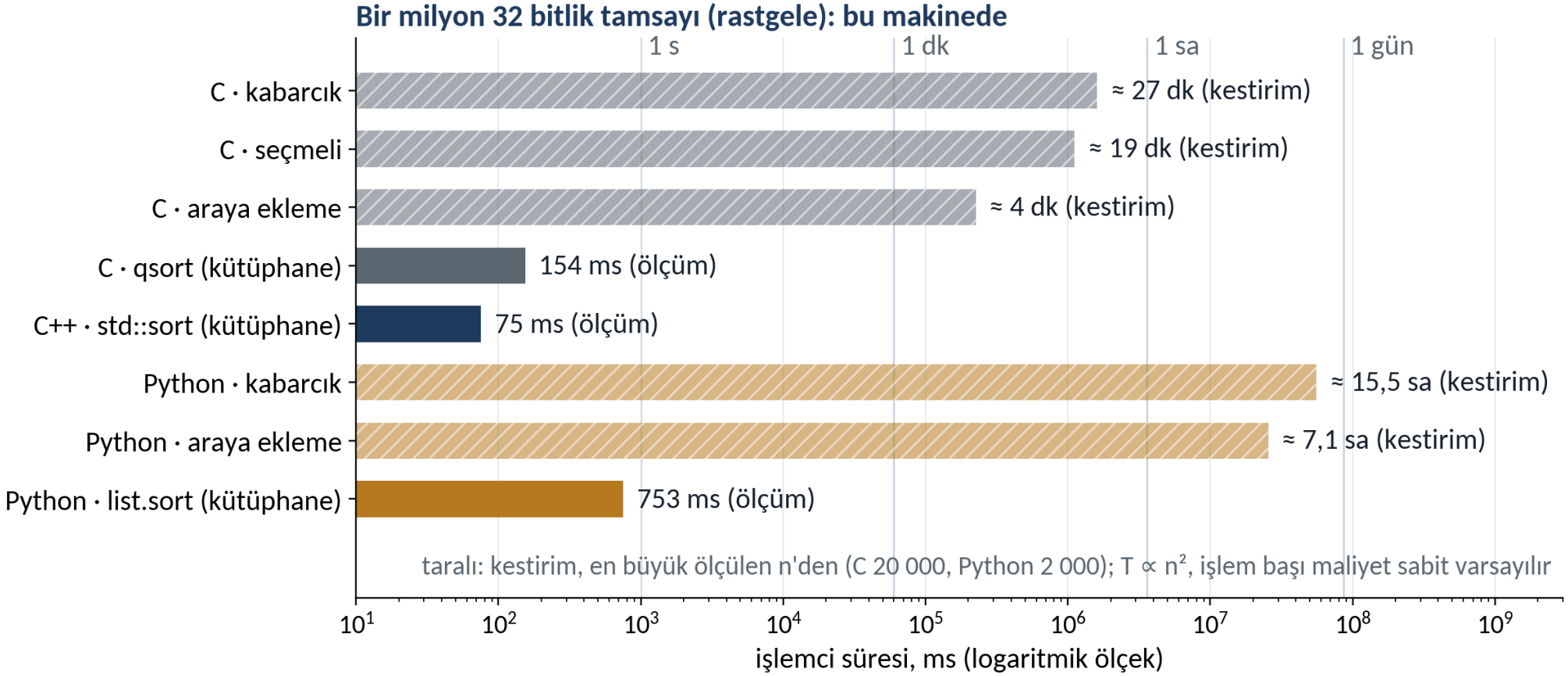
- **S1 (H4).** 16 elemanlı sıralı `{3, 8, 14, …, 95}` dizisinde ikili arama x = 61'i en çok kaç yoklamada bulur? Dizi sıralı olmasaydı?
- **S2 (H4, Örnek 8).** Rastgele 1000 sayıyı tek tek sıralı diziye eklediğimizde toplam kaydırma 253 576 çıktı; $n^2/4$ = 250 000. Rastlantı mı?
- **S3 (AP-1 H10).** n elemanlı dizide en küçüğün indisi kaç karşılaştırma ister? Takası `a[i] = a[j]; a[j] = a[i];` diye yazarsak ne olur?

Yanıtlar: üçü de bu haftanın yapı taşı

- **S1:** 4 yoklama (H4 Örnek 3'ün izleme tablosu). Sırasız dizide ya Θ(n) doğrusal arama ya da **önce sıralama**.
- **S2:** rastlantı değil. Eklenen sayının kaydırması, kendisinden önce gelip ondan büyük olanların sayısıdır; rastgele sırada her çift 1/2 olasılıkla “ters”tir, toplam ≈ n(n − 1)/4. Bu sayıya **ters çevrim** diyeceğiz.
- **S3:** n − 1 karşılaştırma. Geçici değişkensiz yazımda iki eleman da a[j] olur, a[i] kaybolur: takas üç atamadır.
- H4 Örnek 8'in programı aslında **ikili aramalı araya ekleme sıralamasıydı**: az yoklama, çok kaydırma.

Büyük Soru: bir milyon tamsayı, üç algoritma, bir kütüphane

Elinizdeki iskambil kâğıtlarını sıralarken hangi algoritmayı kullanıyorsunuz ve bir milyon tamsayıyı sıralamak için kabarcık sıralaması neden “yanlış yol”dur? Bu makinede, C (-O2): kabarcık ≈ 26,7 dk (kestirim), `qsort` 153,9 n



GERÇEK DÜNYADAN

2003 · 2007

“The bubble sort would be the wrong way to go”

- 14 Kasım 2007, Google, Mountain View: aday Barack Obama'nın konuşmasından sonraki söyleşide Eric Schmidt “standart bir Google mülakat sorusu” sorar: “bir milyon 32 bitlik tamsayıyı sıralamanın en verimli yolu nedir?”
- Obama'nın yanıtı: “kabarcık sıralaması yanlış yol olur.”
- Astrachan (2003): “bubble sort” adı basılı olarak görünüşe göre ilk kez Iverson'ın 1962 kitabında geçer; Java'da dizgilerle kabarcık, araya eklemeden **neredeyse üç kat** yavaş çıkmıştır.

McLaughlin, A. (2007). Candidates at Google: Barack Obama. Google Public Policy Blog. · Astrachan, O. (2003). Bubble sort: An archaeological algorithmic analysis. ACM SIGCSE Bulletin, 35(1), 1–5.

ÖĞRENME ÇIKTILARI

Bu hafta sonunda neler yapabileceksiniz?

ÖÇ 5.1	Seçmeli, kabarcık ve araya ekleme sıralamasını gerçekler; her biri için döngü değişmezini ifade eder.
ÖÇ 5.2	Üç algoritmayı karşılaştırma ve takas sayıları, en iyi/en kötü durum, kararlılık, yerindelik ve uyarlanabilirlik açısından karşılaştırır.
ÖÇ 5.3	Ters çevrim (inversion) kavramıyla araya ekleme sıralamasının çalışma süresini ilişkilendirir.
ÖÇ 5.4	Karşılaştırıcı fonksiyonlarla yapı dizilerini çok anahtarlı ölçütlere göre sıralar (qsort, std::sort/stable_sort, sorted/key).
ÖÇ 5.5	Kararlılığın çok anahtarlı sıralamadaki rolünü deneyle gösterir.

Neden sıralarız? Sıralı veride dört işlem tek geçişe iner

Sıralama problemi: çıktı **sıralı** olmalı ve girdinin bir **permütasyonu** olmalı (hiçbir öğe kaybolmaz, çoğalmaz).

Arama

İkili arama Θ(log n) (H4); sırasız dizide Θ(n).

Yinelenen tespiti

Eşitler yan yana gelir: Kavram 4'te her çifti denetleyen arama 20, komşu taraması 2 karşılaştırma.

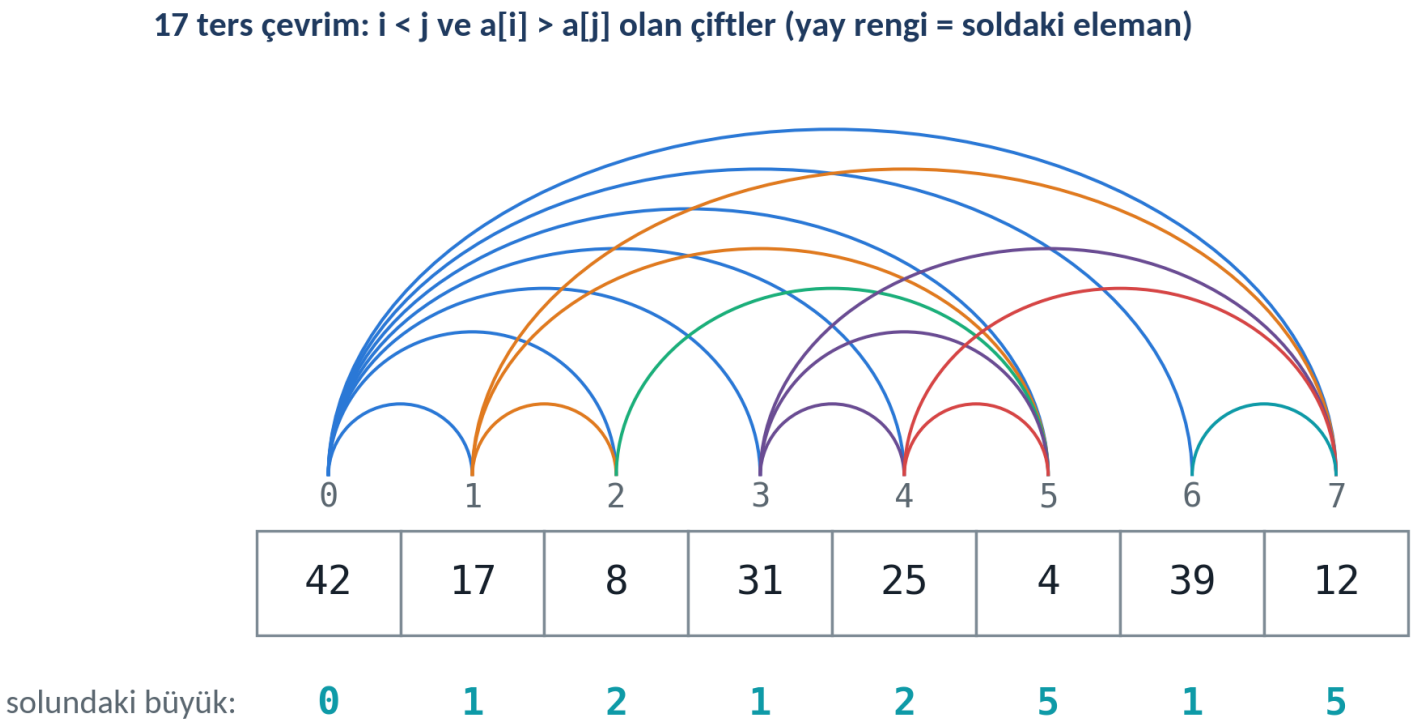
Gruplama

Eşitlerin ardışık bloklarını saymak: değer × sayı tek geçişte.

Birleştirme

İki sıralı dizi tek geçişte birleşir (12 karşılaştırma, en çok 15): H6'nın çekirdeği.

Ters çevrim (inversion): ortak dizide 17 “yanlış sıralı” çift



- i < j ve a[i] > a[j] olan (i, j) çifti.
- Sıralı: 0 · ters sıralı: n(n − 1)/2 · rastgele: ortalama n(n − 1)/4.
- Komşu takası tam **1** ters çevrim giderir → kabarcık takası = T.
- Araya ekleme her öğeyi solundaki büyüklerin üstünden kaydırır → kaydırma = T.

01

Üç temel sıralama algoritması

Aynı dizi, aynı sayaçlar: karşılaştırma · takas · kaydırma

Seçmeli sıralama: kalanların en küçüğünü seç, öneğin sonuna takas et



- Sınıfı boy sırasına dizen öğretmen: kalanların en kısasını bul, sıranın başına çağır.
- i = 0: a[1..7] tarandı, en küçük **a[5] = 4** → a[0] ↔ a[5].
- Bir geçiş = n − 1 − i karşılaştırma + en çok 1 takas.
- Yeşil önek: **sıralı ve dizinin en küçükleri**.

Seçmeli sıralama: i = 1, 2, 3

	0	1	2	3	4	5	6	7	
başlangıç	42	17	8	31	25	4	39	12	
i = 0	**4**	17	8	31	25	**42**	39	12	en küçük a[5] → takas a[0] ↔ a[5] toplam: karşılaştırma 7 · takas 1
i = 1	4	**8**	**17**	31	25	42	39	12	en küçük a[2] → takas a[1] ↔ a[2] toplam: karşılaştırma 13 · takas 2
i = 2	4	8	**12**	31	25	42	39	**17**	en küçük a[7] → takas a[2] ↔ a[7] toplam: karşılaştırma 18 · takas 3
i = 3	4	8	12	**17**	25	42	39	**31**	en küçük a[7] → takas a[3] ↔ a[7] toplam: karşılaştırma 22 · takas 4
i = 4									**? (sizce?)**
i = 5									
i = 6									

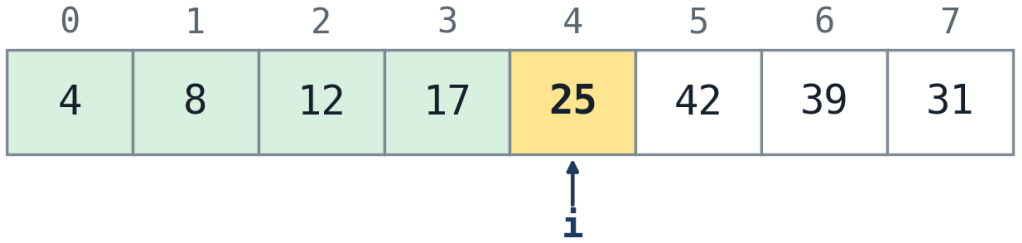
işlenmiş (değişmez bölge) · etkin / yer değiştiren · işlenmemiş

- i = 2 ve i = 3'te en küçük **dizinin sonunda** (12, 17): uzak takas.
- Sayaçlar: karşılaştırma 7 + 6 + 5 + 4 = 22, takas 4.
- Önek her geçişte bir öğe büyür ve bir daha değişmez.

BİR SONRAKİ ADIM HANGİSİ?

i = 4 geçişinde ne olur? Dizi şimdi {4, 8, 12, 17, 25, 42, 39, 31}

i = 4 geçişi başlıyor: a[0..3] yerinde



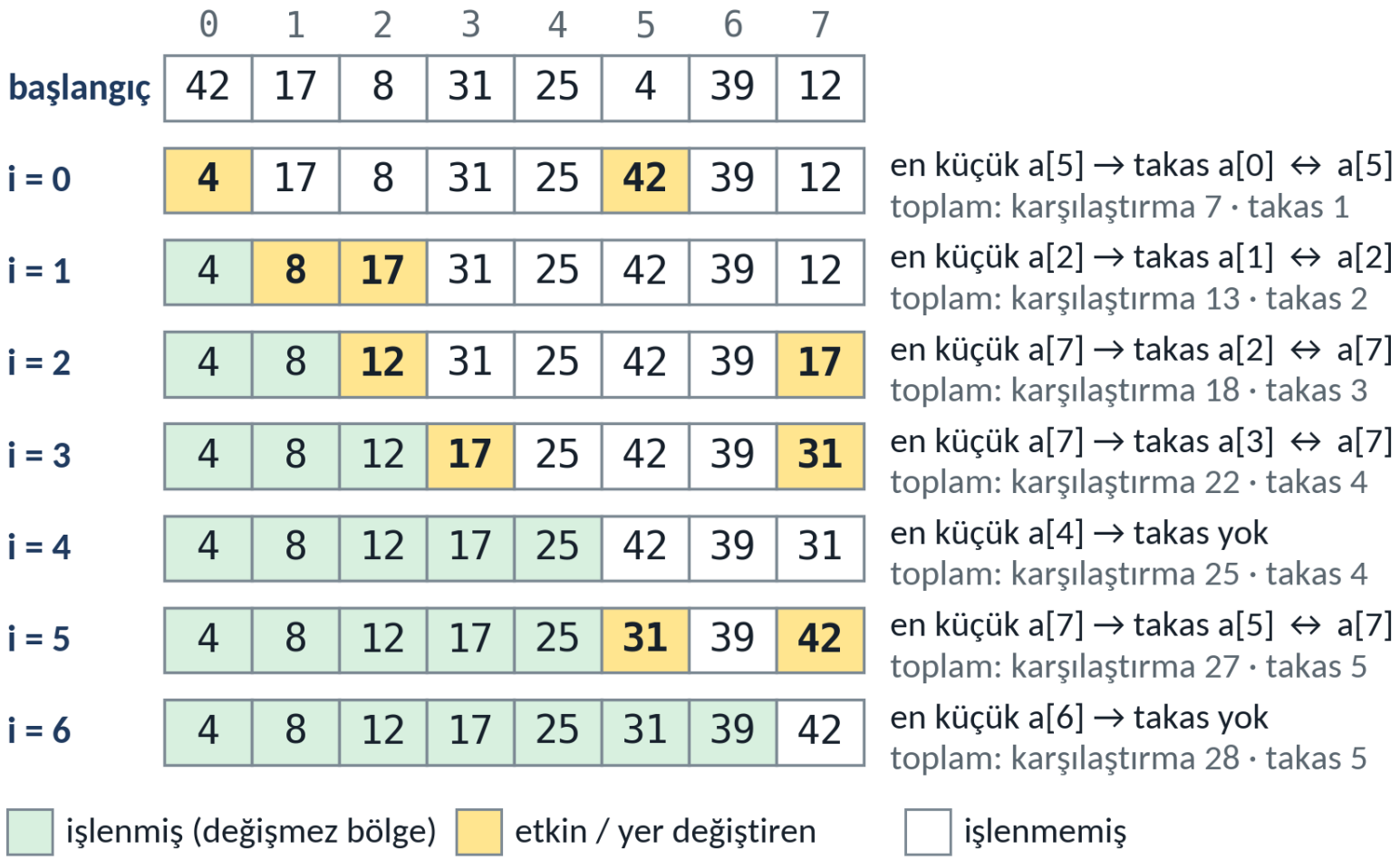
A a[4] ↔ a[7] takas edilir (31 öne gelir)

B En küçük a[4] = 25: zaten yerinde, **takas yapılmaz**

C a[4] ↔ a[5] takas edilir (42 öne gelir)

D Dizi sıralı sayılır, algoritma durur

Seçmeli sıralama: 28 karşılaştırma, 5 takas



- Karşılaştırma **her girdide** n(n − 1)/2 = 28: girdiden bağımsız, uyarlanabilir değil.
- Takas en çok n − 1; burada 5 (i = 4 ve 6'da takas yok).
- Eleman hareketi pahalıysa (büyük kayıt, flaş bellek) en az yazan temel algoritma.

Seçmeli sıralama: sözde kod ve C

```
algoritma SEÇMELİ(a[0..n-1])
  i ← 0'dan n-2'ye kadar her i için
    en ← i
    j ← i+1'den n-1'e kadar her j için
      eğer a[j] < a[en] ise en ← j
    eğer en ≠ i ise
      a[i] ile a[en]'i takas et
```

c/H05_01_secmeli_siralama.c · satır 68–78

C

```c
    for (size_t i = 0; i + 1 < n; i++) {
        size_t en_kucuk = i;
        for (size_t j = i + 1; j < n; j++) {
            karsilastirma++;
            if (a[j] < a[en_kucuk]) {
                en_kucuk = j;
            }
        }
        if (en_kucuk != i) {
            takas(&a[i], &a[en_kucuk]);
            takas_sayisi++;
```

Değişmez: i. geçişin sonunda a[0..i] sıralıdır ve dizinin en küçük i + 1 öğesini içerir.

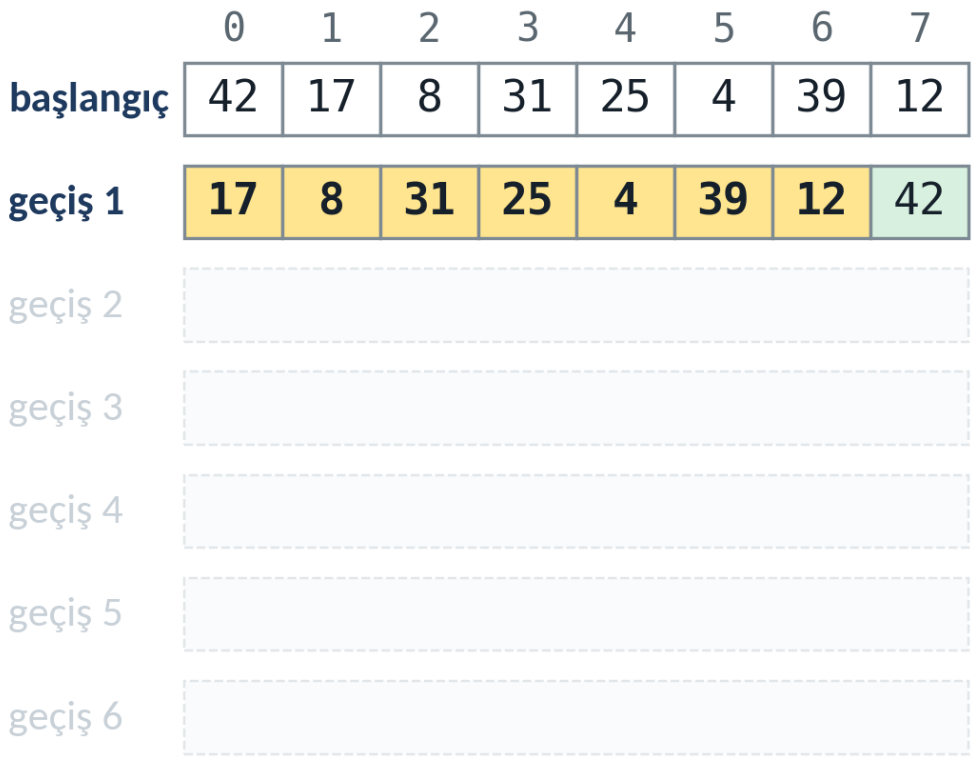
ALGORİTMA KİMLİK KARTI

Seçmeli sıralama *selection sort*

STRATEJİ	GİRDİ ÖNKOŞULU	EN İYİ	ORTALAMA	EN KÖTÜ	EK BELLEK
Kalan kısmın en küçüğünü seç, öneğin sonuna takasla	Yok — karşılaştırılabilir anahtarlar	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$

Özellikler	Kararsız · yerinde · uyarlanabilir değil · en fazla n − 1 takas; Karşılaştırma sayısı her girdide tam n(n−1)/2
Değişmez / doğruluk	i. geçişin sonunda a[0..i] sıralıdır ve dizinin en küçük i + 1 elemanını içerir (a[0..i] ≤ a[i+1..n−1])
Kütüphane karşılığı	C: genel sıralama `qsort` · C++: yapı taşları `std::min_element` + `std::iter_swap` · Python: genel `sorted(a)`, `a.sort()` (yalnızca kâhin)
Ne zaman kullanılır	Yazma/takas çok pahalıysa (en fazla n − 1 takas)
Ne zaman kullanılmaz	Büyük n (her girdide $\Theta(n^2)$ karşılaştırma)

Kabarcık sıralaması: komşu ters sıradaysa takas et



7 karşılaştırma, 7 takas
toplam: karşılaştırma 7 · takas 7

? (sizce?)

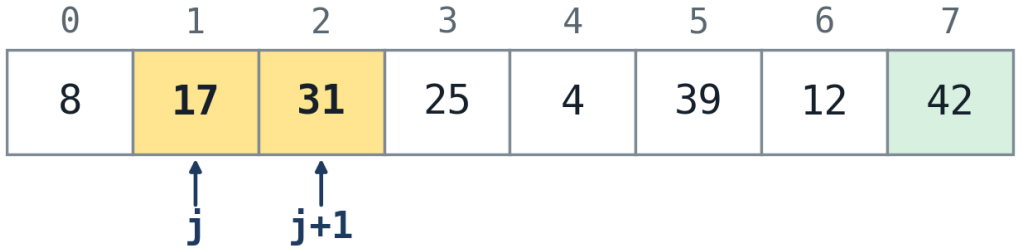
işlenmiş (değişmez bölge) etkin / yer değiştiren işlenmemiş

- Dizi soldan sağa taranır; **komşu** iki öğe ters sıradaysa takas.
- 1. geçiş: 42 her komşusundan büyük → 7 takasla sona “yüzer”.
- Yeşil sonek: yerine oturmuş en büyükler.
- Bir geçişte hiç takas yoksa: **erken çıkış**.

BİR SONRAKİ ADIM HANGİSİ?

2. geçiş, j = 1: dizi {8, 17, 31, 25, 4, 39, 12, 42}. Sıradaki takas hangisi?

geçiş 2: a[0] ↔ a[1] takası az önce yapıldı



A a[0] ↔ a[1] (8, 17)

B a[1] ↔ a[2] (17, 31)

C a[2] ↔ a[3] (31, 25)

D a[3] ↔ a[4] (25, 4)

Kabarcık sıralaması: 6 geçiş, 27 karşılaştırma, 17 takas

	0	1	2	3	4	5	6	7	
başlangıç	42	17	8	31	25	4	39	12	
geçiş 1	**17**	**8**	**31**	**25**	**4**	**39**	**12**	42	7 karşılaştırma, 7 takas toplam: karşılaştırma 7 · takas 7
geçiş 2	**8**	**17**	**25**	**4**	**31**	**12**	39	42	6 karşılaştırma, 4 takas toplam: karşılaştırma 13 · takas 11
geçiş 3	8	17	**4**	**25**	**12**	31	39	42	5 karşılaştırma, 2 takas toplam: karşılaştırma 18 · takas 13
geçiş 4	8	**4**	**17**	**12**	25	31	39	42	4 karşılaştırma, 2 takas toplam: karşılaştırma 22 · takas 15
geçiş 5	**4**	**8**	**12**	17	25	31	39	42	3 karşılaştırma, 2 takas toplam: karşılaştırma 25 · takas 17
geçiş 6	4	8	12	17	25	31	39	42	2 karşılaştırma, 0 takas → dur toplam: karşılaştırma 27 · takas 17

işlenmiş (değişmez bölge) · etkin / yer değiştiren · işlenmemiş

- Takas sayısı = ters çevrim sayısı = **17**: her komşu takası tam bir ters çevrim giderir.
- Dizi 5. geçişte sıralandı; algoritma bunu 6. geçişte hiç takas görmeyince anladı.
- Eşitler takas edilmez (>): **kararlı**.

Tavşanlar ve kaplumbağalar: kabarcığın asimetrisi

tavşan: 8 başta

başla	**8**	1	2	3	4	5	6	7
1.	1	2	3	4	5	6	7	**8**
2.	1	2	3	4	5	6	7	**8**

2 geçiş, 13 karşılaştırma, 7 takas

kaplumbağa: 1 sonda

başla	2	3	4	5	6	7	8	**1**
1.	2	3	4	5	6	7	**1**	8
2.	2	3	4	5	6	**1**	7	8
3.	2	3	4	5	**1**	6	7	8
4.	2	3	4	**1**	5	6	7	8
5.	2	3	**1**	4	5	6	7	8
6.	2	**1**	3	4	5	6	7	8
7.	**1**	2	3	4	5	6	7	8

7 geçiş, 28 karşılaştırma, 7 takas

- Başta duran büyük öğe (tavşan) tek geçişte sona gider: 2 geçiş, 13 karşılaştırma.
- Sonda duran küçük öğe (kaplumbağa) geçiş başına 1 adım: 7 geçiş, 28 karşılaştırma, yalnızca 7 takas.
- Uyarlanabilirlik **zayıf**: tek bir kaplumbağa n − 1 geçiş ister.

Kabarcık sıralaması: sözde kod ve C

```
algoritma KABARCIK(a[0..n-1])
  i ← 0'dan n-2'ye kadar her i için
    takas_oldu ← yanlış
    j ← 0'dan n-2-i'ye kadar her j için
      eğer a[j] > a[j+1] ise
        takas et; takas_oldu ← doğru
    eğer takas_oldu değil ise dur
```

c/H05_06_kahin_testi.c · satır 46–57

C

```c
    for (size_t i = 0; i + 1 < n; i++) {
        int takas_oldu = 0;
        for (size_t j = 0; j + 1 < n - i; j++) {
            if (a[j] > a[j + 1]) {
                takas(&a[j], &a[j + 1]);
                takas_oldu = 1;
            }
        }
        if (!takas_oldu) {
            break;
        }
    }
```

Değişmez: i geçişten sonra son i konum dizinin en büyük i öğesini sıralı olarak içerir.

ALGORİTMA KİMLİK KARTI

Kabarcık sıralaması (erken çıkışlı) *bubble sort (with early exit)*

STRATEJİ	GİRDİ ÖNKOŞULU	EN İYİ	ORTALAMA	EN KÖTÜ	EK BELLEK
Komşu karşılaştırma ve takas; geçişte takas yoksa dur	Yok — karşılaştırılabilir anahtarlar	$O(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$

Özellikler	Kararlı (eşitler takas edilmez) · yerinde · uyarlanabilir (zayıf) · takas sayısı = ters çevrim sayısı
Değişmez / doğruluk	i geçişten sonra son i konum (a[n−i..n−1]) dizinin en büyük i elemanını sıralı olarak içerir
Kütüphane karşılığı	C: yok (elle) · C++: yok · Python: yok
Ne zaman kullanılır	Öğretimde
Ne zaman kullanılmaz	Genel amaçlı sıralama: rastgele veride araya eklemeden yavaştır (bu makinede n = 10 000, C -O2: ≈ 7,3 kat)

Araya ekleme: elinizdeki kâğıtları nasıl sıralıyorsunuz?

1) anahtarı ele al: 12 (solundaki kâğıtlar sıralı)



2) büyükleri birer sağa kaydır: boşluk sola gider



3) boşluğa koy: a[2] = 12



- Soldaki kâğıtlar **sıralı bir el**.
- Sıradaki kâğıt (anahtar) ele alınır, büyükler birer **sağa kaydırılır**.
- Açılan **boşluğa** anahtar konur.
- Seçmeli doğru öğeyi seçer; araya ekleme sıradaki öğenin **doğru yerini** bulur.

Araya ekleme: sıralı önek her adımda bir öğe büyür

	0	1	2	3	4	5	6	7
başlangıç	42	17	8	31	25	4	39	12
i = 1	**17**	42	8	31	25	4	39	12
i = 2	**8**	17	42	31	25	4	39	12
i = 3								
i = 4								
i = 5								
i = 6								
i = 7								

anahtar 17: 1 kaydırma → a[0]
toplam: karşılaştırma 1 · kaydırma 1

anahtar 8: 2 kaydırma → a[0]
toplam: karşılaştırma 3 · kaydırma 3

? (sizce?)

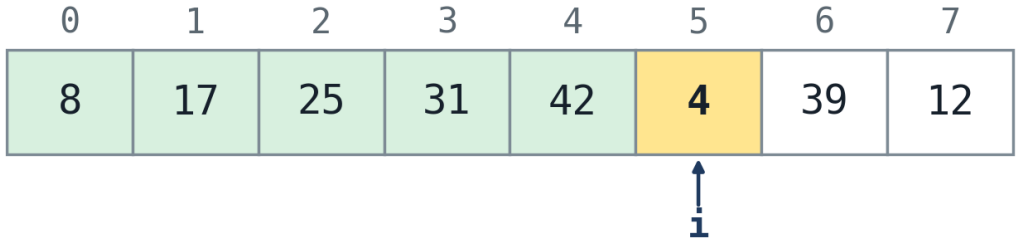
işlenmiş (değişmez bölge) · etkin / yer değiştiren · işlenmemiş

- i = 1: anahtar 17 < 42 → 1 kaydırma, a[0]'a.
- i = 2: anahtar 8 → 2 kaydırma, a[0]'a.
- Önek “ilk i öğenin sıralı hâli”dir, dizinin en küçükleri **değil**.

BİR SONRAKİ ADIM HANGİSİ?

i = 5: dizi {8, 17, 25, 31, 42, 4, 39, 12}, anahtar 4. Kaç kaydırma, anahtar nereye?

i = 5: anahtar = 4, a[0..4] sıralı



A 5 kaydırma; anahtar a[0]'a yerleşir

B 4 kaydırma; anahtar a[1]'e yerleşir

C 1 kaydırma; anahtar a[4]'e yerleşir

D 0 kaydırma; anahtar yerinde kalır

Araya ekleme: 21 karşılaştırma, 17 kaydırma

	0	1	2	3	4	5	6	7	
başlangıç	42	17	8	31	25	4	39	12	
i = 1	**17**	42	8	31	25	4	39	12	anahtar 17: 1 kaydırma → a[0] toplam: karşılaştırma 1 · kaydırma 1
i = 2	**8**	17	42	31	25	4	39	12	anahtar 8: 2 kaydırma → a[0] toplam: karşılaştırma 3 · kaydırma 3
i = 3	8	17	**31**	42	25	4	39	12	anahtar 31: 1 kaydırma → a[2] toplam: karşılaştırma 5 · kaydırma 4
i = 4	8	17	**25**	31	42	4	39	12	anahtar 25: 2 kaydırma → a[2] toplam: karşılaştırma 8 · kaydırma 6
i = 5	**4**	8	17	25	31	42	39	12	anahtar 4: 5 kaydırma → a[0] toplam: karşılaştırma 13 · kaydırma 11
i = 6	4	8	17	25	31	**39**	42	12	anahtar 39: 1 kaydırma → a[5] toplam: karşılaştırma 15 · kaydırma 12
i = 7	4	8	**12**	17	25	31	39	42	anahtar 12: 5 kaydırma → a[2] toplam: karşılaştırma 21 · kaydırma 17

işlenmiş (değişmez bölge) · etkin / yer değiştiren · işlenmemiş

- Kaydırma = ters çevrim = **17**.
- Karşılaştırma = T + (n − 1) − z (z: başa yerleşen anahtar sayısı).
- Süre Θ(n + T): sıralı girdide O(n), ters sıralıda Θ(n²): **uyarlanabilir**.
- Eşitte durur (>): **kararlı**; yeni öğeyi aynı adımla ekler: **çevrimiçi**.

Araya ekleme: sözde kod ve C

```
1  algoritma ARAYA-EKLEME(a[0..n–1])
2    i ← 1'den n–1'e kadar her i için
3      anahtar ← a[i];  j ← i – 1
4      j ≥ 0 ve a[j] > anahtar iken
5        a[j+1] ← a[j];  j ← j – 1
6      a[j+1] ← anahtar
```

c/H05_06_kahin_testi.c · satır 62–70 — C

```c
62      for (size_t i = 1; i < n; i++) {
63          int anahtar = a[i];
64          ptrdiff_t j = (ptrdiff_t)i - 1;
65          while (j >= 0 && a[j] > anahtar) {
66              a[j + 1] = a[j];
67              j--;
68          }
69          a[j + 1] = anahtar;
70      }
```

Değişmez: i. adımın başında a[0..i – 1], girdinin ilk i öğesinin sıralı hâlidir.

ALGORİTMA KİMLİK KARTI

Araya ekleme sıralaması *insertion sort*

STRATEJİ	GİRDİ ÖNKOŞULU	EN İYİ	ORTALAMA	EN KÖTÜ	EK BELLEK
Sıradaki anahtarı, ondan büyükleri bir sağa kaydırarak yerine koy	Yok — karşılaştırılabilir anahtarlar	$O(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$

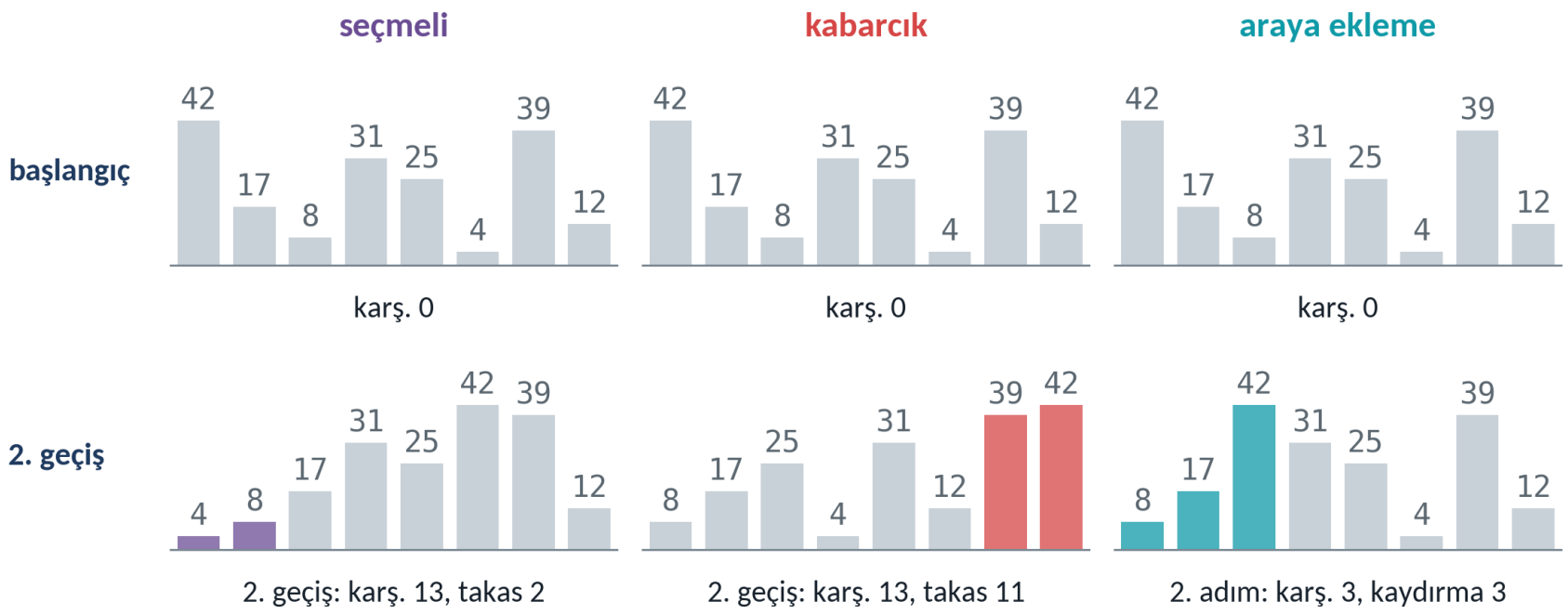
Özellikler	Kararlı · yerinde · uyarlanabilir · çevrimiçi; kaydırma sayısı = ters çevrim sayısı, süre $\propto n$ + ters çevrim; k-sıralı dizide $O(nk)$
Değişmez / doğruluk	i. adımın başında a[0..i−1], girdinin ilk i elemanının sıralı hâlidir
Kütüphane karşılığı	C: kaydırma için `memmove` · C++: sıralı vektöre ekleme `std::upper_bound` + `insert` · Python: sıralı listeye ekleme `bisect.insort`
Ne zaman kullanılır	Küçük diziler (kütüphaneler küçük alt dizilerde kullanır: introsort, Timsort)
Ne zaman kullanılmaz	Büyük ve rastgele veri ($\Theta(n^2)$): birleştirmeli ya da hızlı sıralama (H6)

Üç değişmez, üç farklı söz



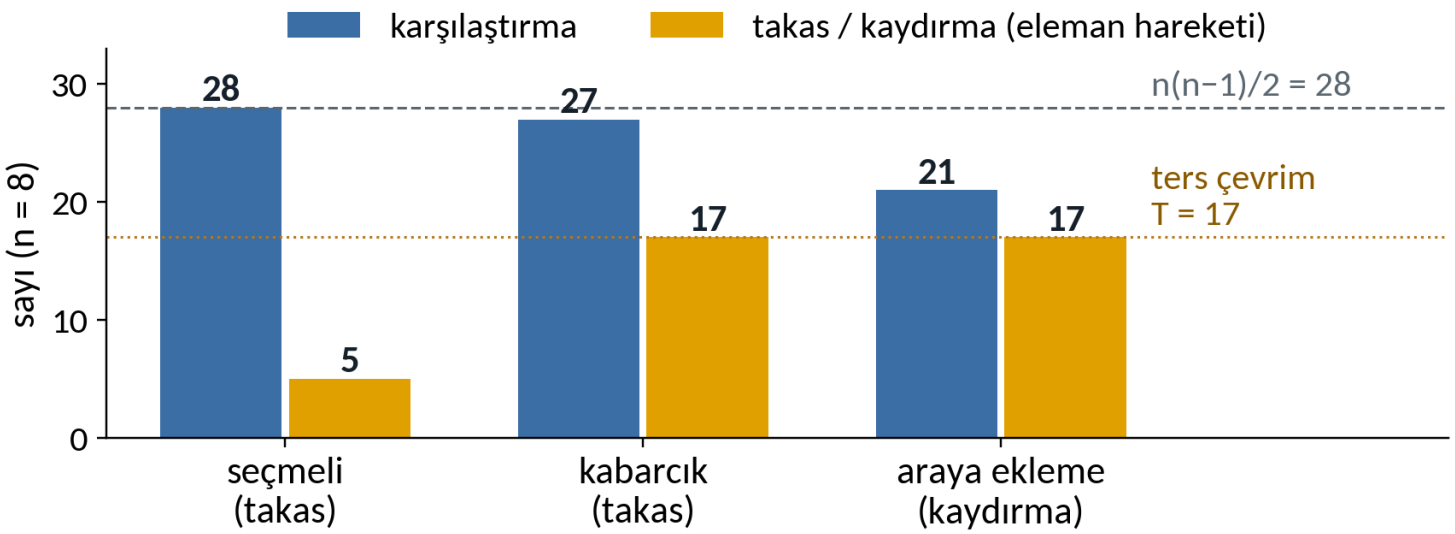
- **Seçmeli:** önek sıralı **ve** en küçükler; bir daha değişmez.
- **Kabarcık:** sonek en büyükler, yerine oturmuş.
- **Araya ekleme:** önek yalnızca ilk i öğenin sıralı hâli.
- Hepsi **başlatma, koruma, sonlanma** ile kanıtlanır.

Aynı anlarda üç algoritma



- Çubuk yüksekliği = değer; renkli çubuklar değişmezin bölgesi.
- İkinci geçiş/adımdan sonra: seçmeli 13 karş./2 takas, kabarcık 13/11, araya ekleme 3/3.
- Web sayfasındaki **sıralama görselleştirici** aynı sayaçlarla adım adım ilerler.

Aynı Θ(n²), farklı iş



- **Seçmeli:** her girdide n(n − 1)/2 karşılaştırma; en çok n − 1 takas.
- **Kabarcık:** en iyi n − 1; takas = T; karşılaştırma ≈ n²/2.
- **Araya ekleme:** en iyi n − 1; kaydırma = T; karşılaştırma ≈ n²/4.
- Kabarcık ve araya ekleme aynı sayıda öğe hareket ettirir; araya ekleme yarı karşılaştırmayla.

Üç dilde: seçmeli sıralamanın gövdesi

H05_01_secmeli_siralama.c · satır 68–78 — C

```c
    for (size_t i = 0; i + 1 < n; i++) {
        size_t en_kucuk = i;
        for (size_t j = i + 1; j < n; j++) {
            karsilastirma++;
            if (a[j] < a[en_kucuk]) {
                en_kucuk = j;
            }
        }
        if (en_kucuk != i) {
            takas(&a[i], &a[en_kucuk]);
            takas_sayisi++;
```

H05_01_secmeli_siralama.cpp · satır 67–77 — C++

```cpp
    for (std::size_t i = 0; i + 1 < n; i++) {
        std::size_t en_kucuk = i;
        for (std::size_t j = i + 1; j < n; j++) {
            karsilastirma++;
            if (a[j] < a[en_kucuk]) {
                en_kucuk = j;
            }
        }
        if (en_kucuk != i) {
            std::swap(a[i], a[en_kucuk]);
            takas_sayisi++;
```

H05_01_secmeli_siralama.py · satır 49–57 — Python

```python
    for i in range(n - 1):
        en_kucuk = i
        for j in range(i + 1, n):
            karsilastirma += 1
            if a[j] < a[en_kucuk]:
                en_kucuk = j
        if en_kucuk != i:
            a[i], a[en_kucuk] = a[en_kucuk], a[i]
            takas_sayisi += 1
```

Merak: Shell sıralaması önce uzaktakileri yaklaştırır



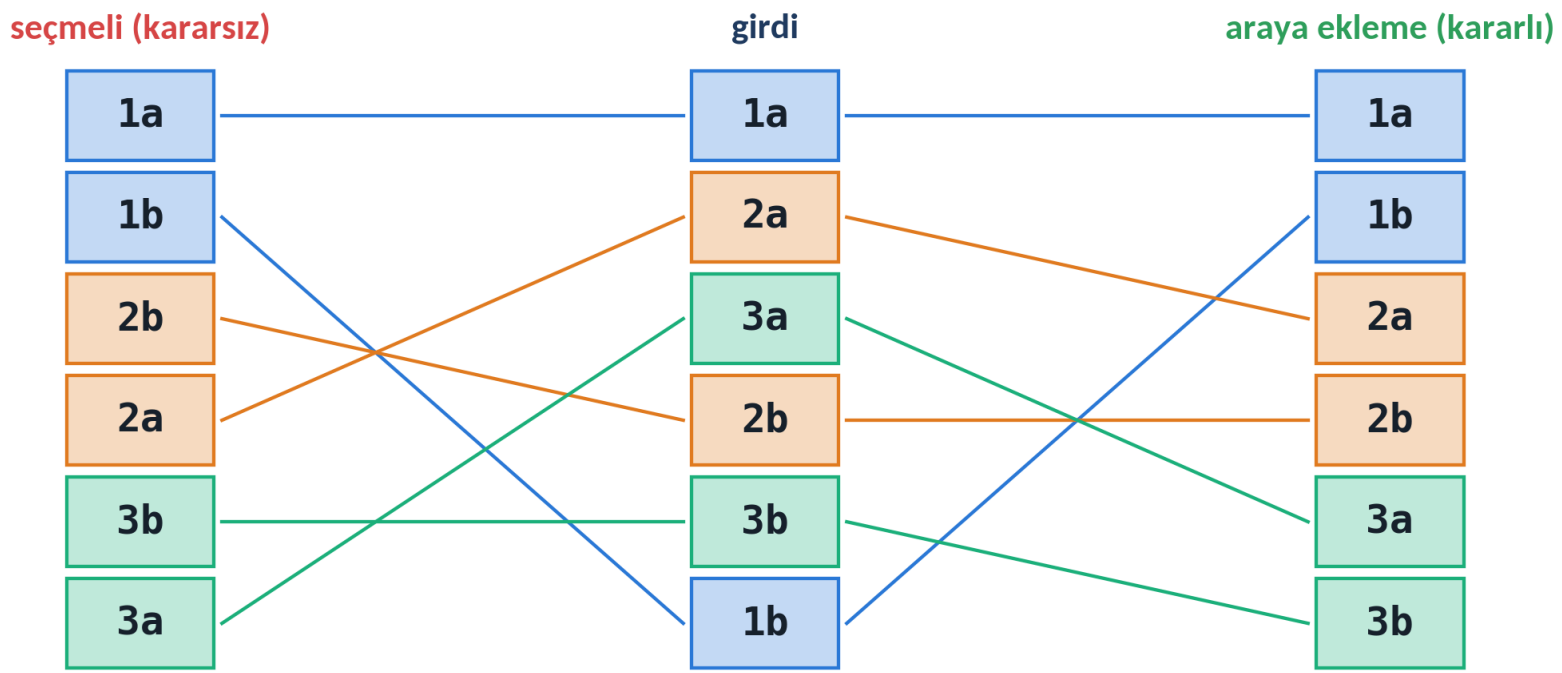
- h = 4, 2, 1: aralığı h olan alt dizilerde araya ekleme (h-sıralama); son geçiş h = 1.
- n = 5 000 rastgele: Ciura 87 458, Knuth 101 009, araya ekleme 6 224 400 karşılaştırma.
- Kararsız: uzak kaydırmalar eşitleri birbirinin üstünden atlatır.

02

Kayıtlar, karşılaştırıcılar, kütüphaneler

Kararlılık · qsort sözleşmesi · çok anahtar · Türkçe alfabe · dolaylı sıralama

Kararlılık: eşit anahtarlıların sırası korunuyor mu?



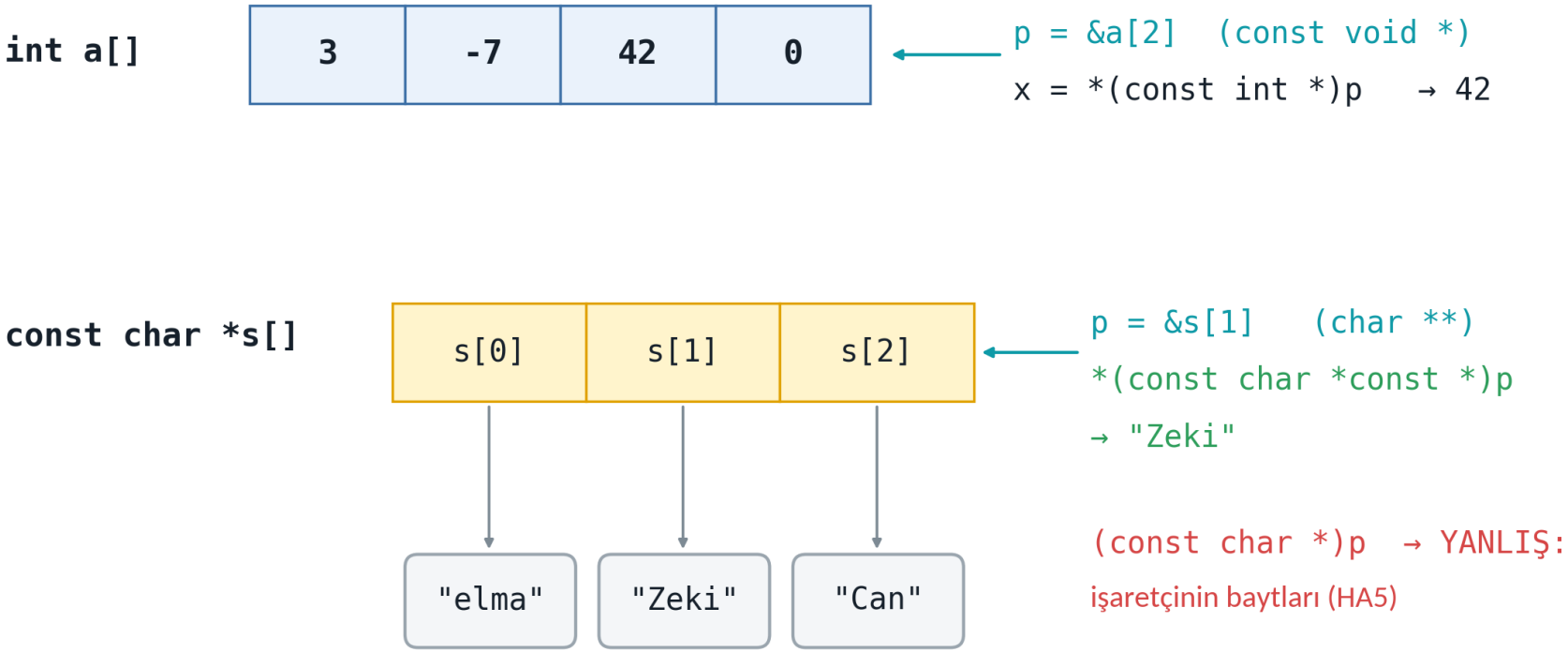
bozulan sıra: 2b önce, 2a sonra; 3b önce, 3a sonra

sayı = anahtar, harf = girdideki sıra; aynı renkli çizgiler kesişiyorsa eşitlerin sırası bozulmuştur

- Sayı = anahtar, harf = girdideki sıra.
- **Seçmeli:** uzak takas 2b'yi 2a'nın, 3b'yi 3a'nın önüne atlatır → kararsız.
- **Araya ekleme:** eşitte durur → kararlı. Kabarcık: eşitte takas etmez → kararlı.
- Kararlılık bir **güvencedir**; tek bir girdide gözlenen davranış değil.

qsort'a “hangisi önce?” diye sormak

qsort(taban, n, boyut, kars): kars(p, q) iki ÖĞENİN ADRESİNİ alır



- `qsort(taban, n, boyut, kars)`: algoritma öğenin türünü bilmez.
- `kars(p, q)`: p ve q öğelerin **adresidir** (`const void *`).
- Sonuç < 0: p önce · 0: eşit · > 0: q önce.
- Dizgi dizisinde öğe bir `char *` → p aslında `char **`.

KOD · ÖRNEK 7

Taşmaya güvenli karşılaştırıcı: (x > y) − (x < y)

c/H05_07_qsort_arayuzu.c · satır 20–31

C

```c
static int int_artan(const void *p, const void *q)
{
    int x = *(const int *)p;
    int y = *(const int *)q;
    return (x > y) - (x < y);   /* -1, 0, 1; taşma yok */
    /* YANLIŞ: return x - y;  x = INT_MAX, y = -1 için taşar (UB) */
}

static int int_azalan(const void *p, const void *q)
{
    return int_artan(q, p);     /* bağımsız değişkenlerin yeri değişir */
}
```

ÇIKTI · ÖRNEK 7 (1. BÖLÜM)

```
1) int dizisi: qsort + taşmaya güvenli karşılaştırıcı
   girdi : 3 -7 2147483647 0 -2147483648 42 -1 7
   artan : -2147483648 -7 -1 0 3 7 42 2147483647
   azalan: 2147483647 42 7 3 0 -1 -7 -2147483648
   neden x - y değil? INT_MAX - (-1) = 2147483648 > INT_MAX = 2147483647
```

Çok anahtar: ortalama ↓, soyad ↑, ad ↑

girdi (kayıt sırası)		1) ad ↑		2) soyad ↑ (kararlı)		3) ortalama ↓ (kararlı)	
101 Ayşe Yılmaz	78,50	107 Ali Yılmaz	78,50	108 Deniz Arslan	90,00	108 Deniz Arslan	**90,00**
102 Mehmet Kaya	65,25	101 Ayşe Yılmaz	78,50	106 Burak Demir	65,25	104 Can Yılmaz	**90,00**
103 Zeynep Demir	78,50	106 Burak Demir	65,25	110 Selin Demir	78,50	110 Selin Demir	**78,50**
104 Can Yılmaz	90,00	104 Can Yılmaz	90,00	103 Zeynep Demir	78,50	103 Zeynep Demir	**78,50**
105 Elif Kaya	78,50	108 Deniz Arslan	90,00	105 Elif Kaya	78,50	105 Elif Kaya	**78,50**
106 Burak Demir	65,25	105 Elif Kaya	78,50	109 Emre Kaya	65,25	107 Ali Yılmaz	**78,50**
107 Ali Yılmaz	78,50	109 Emre Kaya	65,25	102 Mehmet Kaya	65,25	101 Ayşe Yılmaz	**78,50**
108 Deniz Arslan	90,00	102 Mehmet Kaya	65,25	107 Ali Yılmaz	78,50	106 Burak Demir	**65,25**
109 Emre Kaya	65,25	110 Selin Demir	78,50	101 Ayşe Yılmaz	78,50	109 Emre Kaya	**65,25**
110 Selin Demir	78,50	103 Zeynep Demir	78,50	104 Can Yılmaz	90,00	102 Mehmet Kaya	**65,25**

son sıra = tek karşılaştırıcı (ortalama ↓, soyad ↑, ad ↑): eşit ortalamalılar önceki geçişin (soyad, ad) sırasını korur

- **Tek karşılaştırıcı:** anahtarları önem sırasıyla dene; ilk farklı karar verir.
- **Ardışık geçişler:** önce en önemsiz (ad), en son en önemli (ortalama) anahtara göre **kararlı** sırala.
- Kararsız geçiş önceki sırayı bozar (Örnek 10, C).

Üç dilde: çok anahtarlı karşılaştırıcı

H05_10_cok_anahtarli.c · satır 43–51 — C

```c
    int k = ortalama_azalan(x, y);
    if (k != 0) {
        return k;
    }
    k = soyad_artan(x, y);
    if (k != 0) {
        return k;
    }
    return ad_artan(x, y);
```

H05_10_cok_anahtarli.cpp · satır 46–52 — C++

```cpp
    if (x.ortalama != y.ortalama) {
        return x.ortalama > y.ortalama;
    }
    if (x.soyad != y.soyad) {
        return x.soyad < y.soyad;
    }
    return x.ad < y.ad;
```

H05_10_cok_anahtarli.py · satır 28–30 — Python

```python
def cok_anahtarli(o):
    """Tek anahtar: demet (tuple) sözlük sırasıyla karşılaştırılır"""
    return (-o.ortalama, o.soyad, o.ad)
```

C ve C++'ta anahtarlar sırayla denenir (`if (k != 0) return k;`); Python'da tek bir demet anahtar: azalan ölçüt için eksi işareti.

strcmp Türkçe bilmez: “Çelik”, “Zeki”den sonra gelir

strcmp bayt bayt karşılaştırır; UTF-8'de Ç (C3 87) ve Ş (C5 9E) ASCII harflerinden (≤ 7A) büyüktür

bayt sırası (strcmp)

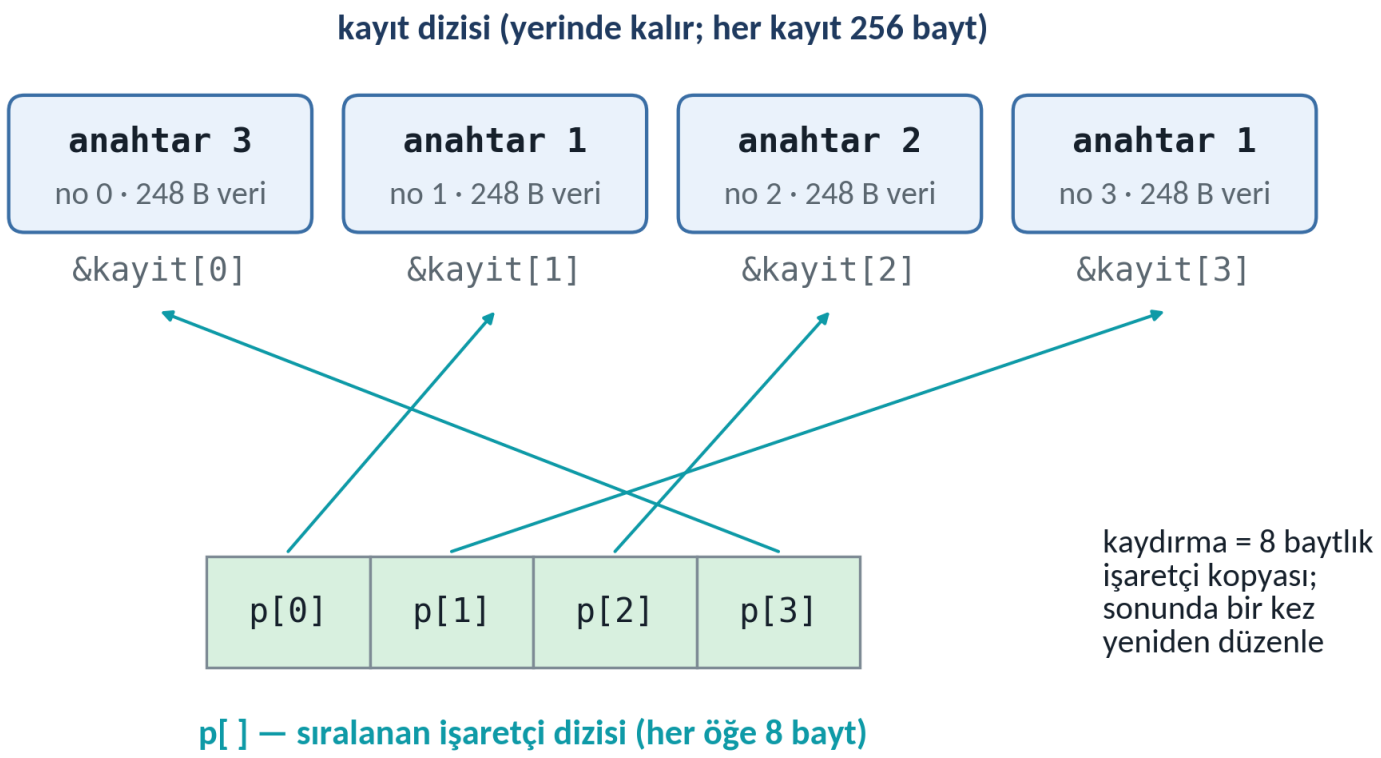
1. Can	43 61 6E
2. Demir	44 65 6D …
3. Zeki	5A 65 6B …
4. Çelik	C3 87 65 …
5. Şahin	C5 9E 61 …

Türkçe alfabe sırası

1. Can	43 61 6E
2. Çelik	C3 87 65 …
3. Demir	44 65 6D …
4. Şahin	C5 9E 61 …
5. Zeki	5A 65 6B …

UTF-8'de Ç, Ö, Ü, Ş, Ğ, İ iki bayttır ve ilk baytları bütün ASCII harflerinden büyüktür. Çözüm: kod noktasına çöz + Türkçe sıra tablosu (Mini-OBS 2 v2.4).

Kaputun Altında: kayıtları değil işaretçileri taşımak



- 256 baytlık kayıtlar yerinde kalır; 8 baytlık işaretçiler sıralanır.
- Örnek 12 (n = 1000): kopyalanan bayt oranı 28,3.
- Süre (C, n = 5 000, tek çalıştırma): 55,9 ms → 8,8 ms.
- Python listesi zaten başvuru tutar: kazanç yok.

Örnek 12 (n = 1000): kopyalanan bayt 61 829 120 (kayıtlar) ↔ 2 188 160 (işaretçiler + düzenleme), 28,3 kat

Kaputun Altında: kaydırmayı blok olarak yapmak (memmove)

kavram/H05_K01_memmove_kaydirma.c · satır 36–50

C

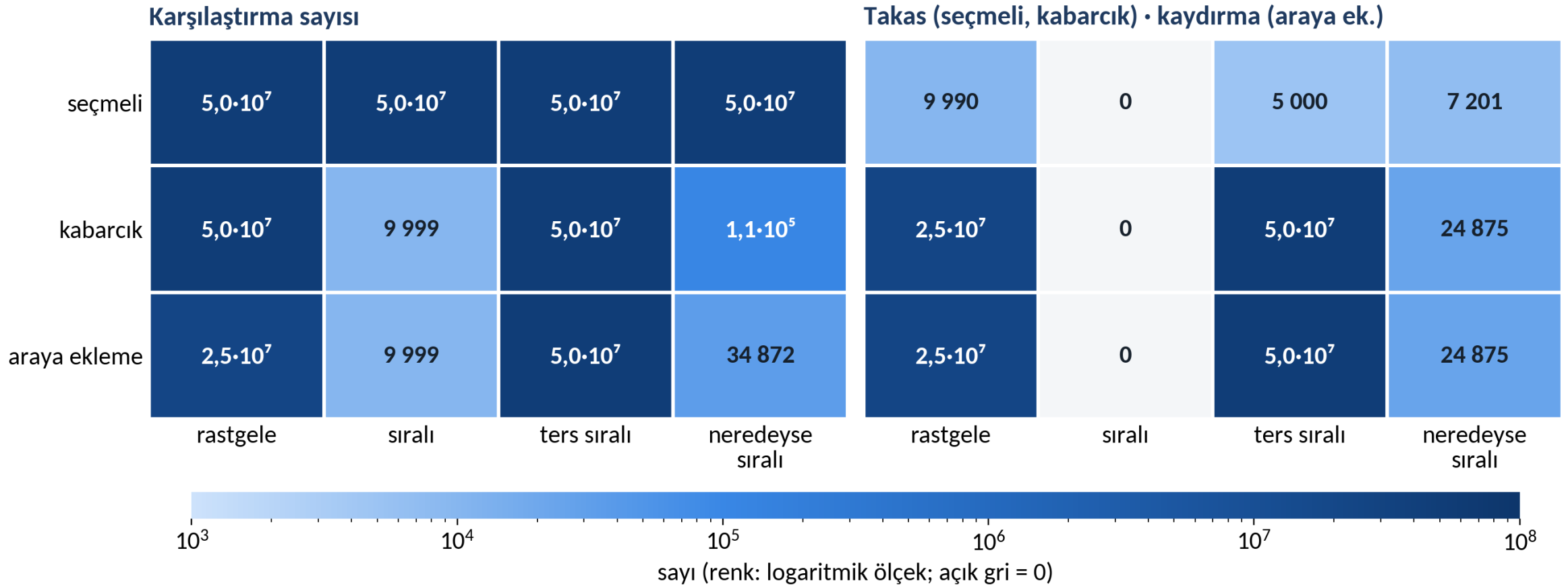
```c
/* Blok: önce yeri bul, sonra a[yer..i-1]'i tek memmove ile kaydır */
static void araya_ekleme_memmove(int *a, size_t n)
{
    for (size_t i = 1; i < n; i++) {
        int anahtar = a[i];
        size_t yer = i;
        while (yer > 0 && a[yer - 1] > anahtar) {
            yer--;                          /* yalnızca karşılaştır */
        }
        /* bölgeler örtüşür: memcpy değil memmove */
        memmove(&a[yer + 1], &a[yer], (i - yer) * sizeof a[0]);
        kaydirma += (long long)(i - yer);
        a[yer] = anahtar;
    }
}
```

ÇIKTI · KAVRAM 1, 20 000 ÖĞE (SÜRE: BU MAKİNEDE, TEK ÇALIŞTIRMA)

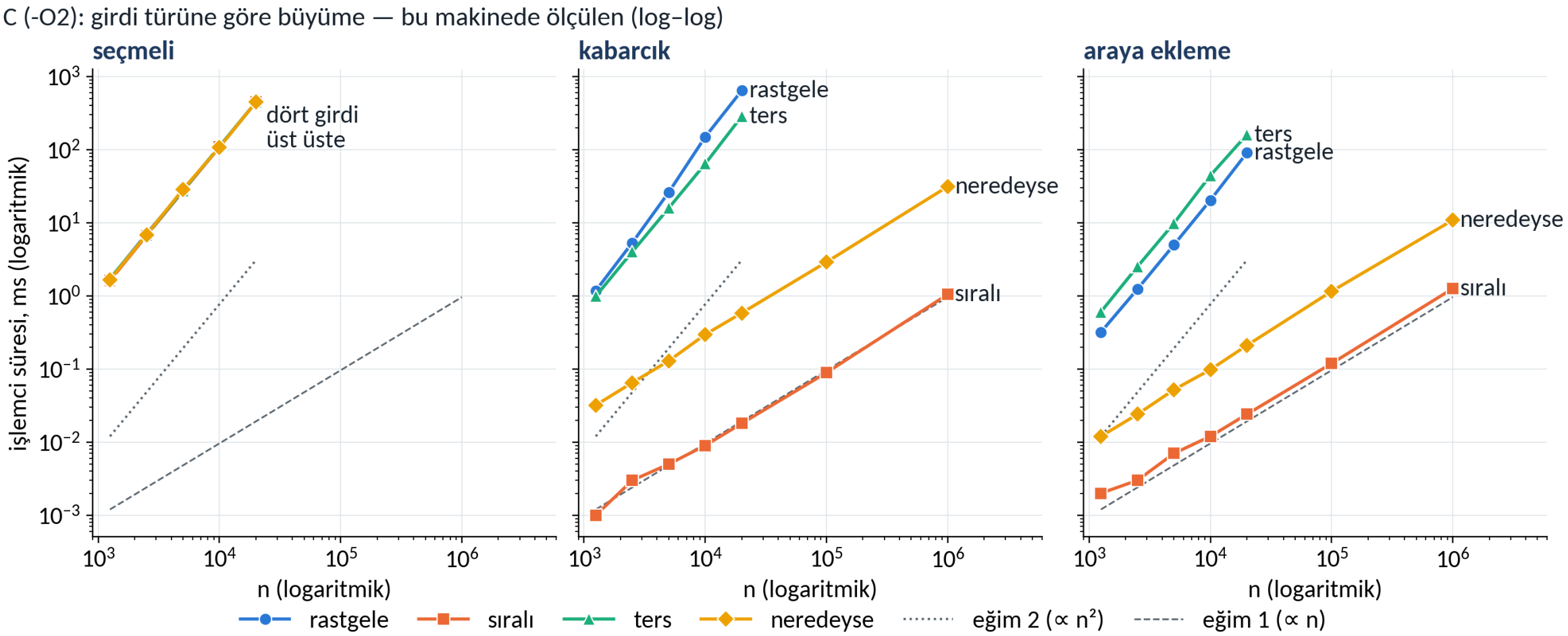
```
rastgele: kaydırma öğe öğe = 100491749, memmove = 100491749; aynı: evet
rastgele: öğe öğe 90.289 ms, memmove 53.004 ms (oran 1.70)   (stderr)
```

Haftanın Deneyi: algoritma × girdi türü (sayaçlar)

Algoritma × girdi türü, n = 10 000 (sayılar üç dilde birebir aynı)



Süre ~ n: karesel durumlar eğim 2, sıralı girdi eğim 1



HATA AVCISI

Koşulun sırası: önce sınır, sonra öğe

hata_avcisi/H05_HA2_kosul_sirasi.c · satır 20–24 · KASITLI HATA

C

```c
        int anahtar = a[i];
        ptrdiff_t j = (ptrdiff_t)i - 1;
        while (a[j] > anahtar && j >= 0) {   /* HATA: koşul sırası */
            a[j + 1] = a[j];
            j--;
```

GÖZLENEN (OLAĞAN DERLEME, SONRA ASAN; ARDINDAN PYTHON)

```
sonuç: 4 8 12 17 25 31 39 42      ← olağan derleme: uyarı yok, çıktı doğru görünüyor
SUMMARY: AddressSanitizer: heap-buffer-overflow H05_HA2_kosul_sirasi.c:22 in araya_ekleme_sirala
karşılaştırma = 24, a[-1] okuması = 3      ← Python: hata yok; a[-1] son öğedir
```

hata_avcisi/H05_HA2_kosul_sirasi_duzeltilmis.c · satır 15–19 · düzeltilmiş

C

```c
        int anahtar = a[i];
        ptrdiff_t j = (ptrdiff_t)i - 1;
        while (j >= 0 && a[j] > anahtar) {   /* önce sınır, sonra değer */
            a[j + 1] = a[j];
            j--;
```

HATA AVCISI

Fark ile karşılaştırma: return x – y taşar

hata_avcisi/H05_HA3_fark_karsilastirici.c · satır 20–24 · KASITLI HATA

C

```c
20  static int int_artan(const void *p, const void *q)
21  {
22      int x = *(const int *)p;
23      int y = *(const int *)q;
24      return x - y;                   /* HATA: taşabilir */
```

GÖZLENEN (OLAĞAN DERLEME, SONRA UBSAN)

```
int   : 0 3 7 2147483647 -2147483648 -1      ← uyarı yok; INT_MAX ortada
double: 0.75 0.25 0.50 0.10 0.30 1.50 2.50      ← (int)(x - y): 0,1 ile 0,3 “eşit”
runtime error: signed integer overflow: 2147483647 - -1 cannot be represented in type 'int'
```

hata_avcisi/H05_HA3_fark_karsilastirici_duzeltilmis.c · satır 12–16 · düzeltilmiş

C

```c
12  static int int_artan(const void *p, const void *q)
13  {
14      int x = *(const int *)p;
15      int y = *(const int *)q;
16      return (x > y) - (x < y);
```

Hata Avcısı: belirti çoğu zaman sayaçta ya da güvencede

İç döngüde takas

Sonuç doğru, ama ters sıralı n = 1000'de 499 500 takas: artık seçmeli değil.

qsort'a sizeof(a)

`a` işaretçiyse öğe 8 bayt sanılır: dizinin dışına yazılır (tanımsız davranış).

Yanlış imza

`(const char *, const char *)`: GCC 13'te uyarı, GCC 14'te hata.

Kararlılığa güvenmek

qsort eşitlerin sırasını belirtmez; `std::sort` 16 öğede korudu, 24 öğede 10 çift bozdu.

strcmp ile Türkçe

Ç, Ö, Ü, Ş, İ listenin sonuna düşer.

x = x.sort()

`list.sort()` yerinde sıralar ve `None` döndürür → TypeError.

Dil Penceresi: aynı algoritma, üç dil

	C	C++	Python
Takas	geçici değişkenli takas(int *, int *)	std::swap(a[i], a[j])	a[i], a[j] = a[j], a[i]
Kütüphane sıralaması	qsort: algoritma ve kararlılık belirtilmez	std::sort kararsız; std::stable_sort kararlı	sorted / list.sort: kararlı
Karşılaştırma	int kars(const void *, const void *)	bool yüklem: x, y'den önce mi?	key= anahtar, reverse=True
Çok anahtar	if (k != 0) return k; zinciri	std::tie(…) < std::tie(…)	demet: key=lambda o: (-o.ort, o.soyad)
Yerinde mi?	qsort yerinde	std::sort yerinde	a.sort() yerinde, None; sorted(a) yeni liste
−1'e inen sayaç	ptrdiff_t j	std::ptrdiff_t j	int (işaretli); ama a[-1] geçerli!



MÜHENDİS GÖZÜYLE

Kütüphaneyi kullan, ama güvencesini oku

- Üç soru: **kararlılık** güvencesi var mı? **karmaşıklık** güvencesi var mı? **karşılaştırıcı sözleşmesi** nedir?
- C `qsort`: üçü de yok ya da sınırlı; eşitlerin sırası belirtilmemiş (C17 7.22.5.2).
- C++ `std::sort`: O(n log n) karşılaştırma, kararlılık yok; yüklem katı zayıf sıralama (`<`, `<=` değil).
- Python: kararlılık belgeli güvence.
- Kâhin ile test ederken ölçütü **tam** yapın: son anahtar öğrenci numarası.

MERAK KUTUSU

Karşılaştırarak sıralamanın alt sınırı: Ω(n log n)

- n öğenin **n!** sıralaması var; algoritma doğru olanı seçmeli.
- Her karşılaştırma olasılıkları en çok **ikiye** böler → en az $\log_2(n!) \approx n \log_2 n$ karşılaştırma.
- Bu haftanın $\Theta(n^2)$ algoritmaları sınırın çok üstünde; H6'nın algoritmaları sınıra ulaşır.
- H7'nin sayma ve taban sıralaması karşılaştırma yapmaz: sınırın dışında.



MİNİ-OBS 2 PROJESİ

Mini-OBS 2 v2.4: çok ölçütlü sıralı listeler

- Mini-OBS'ye çok ölçütlü sıralı listeler ekliyoruz: başarı sırası (YANO ↓, eşitlikte soyad ↑, ad ↑), Türkçe alfabe sırasıyla soyad listesi ve ders ortalaması listeleri.
- Üç algoritma qsort arayüzünde: `(void *taban, size_t n, size_t boyut, kars, Sayac *sy)`.
- Türkçe karşılaştırıcı `tr_karsilastir`: ÇELİK < DEMİR, İREM < JALE, ÖZTÜRK < PEKER, ŞEN < ÜNAL …
- Kâhin: `qsort` + numara (tam ölçüt); kararlılık testi; ölçüm raporu.
- Sayaç farkı: proje **taşıma** sayar (takas = 3 taşıma); ders notunda takas ve kaydırma ayrı.

İLERİ BAKIŞ

Bu haftanın konuları nerede derinleşiyor?

- **H6 · melez sıralama:** birleştirmeli ve hızlı sıralama küçük alt dizileri araya eklemeye devreder (introsort, Timsort).
- **H6 · ters çevrim sayımı:** birleştirme adımına bir sayaç ekleyerek O(n log n); bu haftanın $\Theta(n^2)$ sayımı orada kâhin olacak.
- **H7 · taban sıralaması:** en önemsiz basamaktan başlayan geçişler ancak **kararlıysa** doğru; Örnek 10'un karşılaştırmasız sürümü.

Özet: haftanın kavram haritası



- Seçmeli: her girdide n(n − 1)/2 karşılaştırma, en çok n − 1 takas; kararsız.
- Kabarcık: takas = ters çevrim; kararlı; pratikte en yavaşı.
- Araya ekleme: kaydırma = ters çevrim; kararlı, uyarlanabilir, çevrimiçi.
- Karşılaştırıcı taşmamalı; güvenceyi belgede bulun.

ÇIKIŞ BİLETİ

Dersten çıkmadan önce üç soru

1 {3, 1, 2} dizisini erken çıkışlı kabarcık sıralaması kaç geçişte, kaç karşılaştırma ve kaç takasla sıralar?

2 Seçmeli sıralama neden kararsızdır? Üç öğelik bir karşı örnek verin.

3 `return *(const int *)p - *(const int *)q;` hangi girdide yanlış sıralar? Nasıl düzeltirsiniz?

Hazırlık: bu haftadan sonra

- **Okuma:** ders notu Bölüm 3 (kavramlar), 4 (üç algoritma) ve 7 (Hata Avcısı).
- **Alıştırmalar:** 5.1 (elle izleme), 5.6–5.7 (değişmez), 5.8 (kararlılık hatası), 5.12 (kokteyl), 5.13 (ikili aramalı araya ekleme).
- **Web sayfası:** sıralama görselleştirici, “bir sonraki takas hangisi?”, kararlılık gösterici, karşılaştırıcı oyun alanı; quiz.
- **Atölyeler (ders notu 11.8–11.10):** insan sıralama makinesi (kartlar + sayaç aracı), sıralama laboratuvarı (algoritma × girdi × dil), karşılaştırıcı atölyesi (beş ölçüt).
- **Proje:** Mini-OBS 2 v2.4 — Türkçe karşılaştırıcı ve kâhin testi.
- **Gelecek hafta (H6):** böl ve fethet: birleştirmeli ve hızlı sıralama, O(n log n).